HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

# CAA - Mini-projet

Ransomware post-quantique

**Dylan Oliveira Ramos**

12-12-2025

# 1. Description du ransomware

Nous avons un client (ordinateur de la victime) et un serveur (contrôlé par l'attaquant), le serveur possède une paire de clés publique/privée pour l'établissement d'un secret partagé et une pour signer les messages. Les deux clés publiques sont intégrées au ransomware (client). Pour des raisons de simplicité, les deux entités sont exécutées dans le même programme.

Le programme propose les options suivantes :
1. `Encrypt` : pour chiffrer tous les fichiers d'un dossier choisi.
2. `Pay` : pour payer la rançon et pouvoir déchiffrer tous les fichiers.
3. `Decrypt one file` : pour déchiffrer un fichier spécifique et payer une plus petite rançon.
4. `Change password` : pour changer le mot de passe utilisé pour tout déchiffrer.

## 1.1. Niveau de sécurité choisi

Le ransomware utilise le niveau de sécurité **V**, qui offre une sécurité au moins aussi forte que AES-256.

## 1.2. Communication entre le client et le serveur

Au lancement du programme, le client et le serveur établissent un secret partagé en utilisant l'algorithme **Kyber-1024**. Ce secret partagé est ensuite dérivé avec **HKDF** pour obtenir une clé symétrique utilisée pour chiffrer les communications entre le client et le serveur avec **AES-256-GCM**.

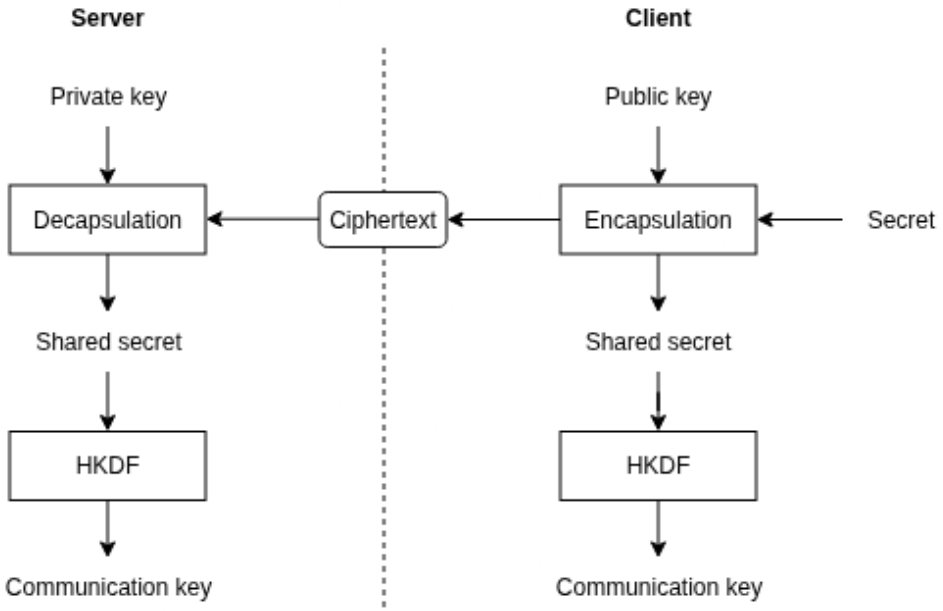


Fig. 1. – Établissement de la clé symétique pour la communication entre le client et le serveur.

Paramètres utilisés pour **Kyber-1024** :
- Taille de la clé publique : 1568 bytes.
- Taille de la clé privée : 3168 bytes.

Paramètres utilisés pour **HKDF** :
- Algorithme de hachage : SHA-256.
- Taille de la clé dérivée : 32 bytes (pour être compatible avec AES-256).
- Sel : aucun.

Paramètres utilisés pour **AES-256-GCM** :
- Taille de la clé : 32 bytes.
- Taille du nonce : 12 bytes.
- Taille du tag : 16 bytes.

Cette architecture est résistante aux attaques post-quantiques car

## 1.3. Chiffrement des fichiers

Lors du choix de l'option `Encrypt`, le client :
1. Génère une `Master Key` aléatoire, la chiffre avec la clé publique du serveur et lui envoie.
2. Génère une `File Key` aléatoire pour chaque fichier du dossier et chiffre chaque fichier avec AES-GCM.
3. Chiffre chaque `File Key` avec AES-KW en utilisant la `Master Key` .
4. Dérive une clé avec Argon2id à partir d'un mot de passe aléatoire d'un dictionnaire et chiffre la `Master Key` avec AES-KW en utilisant cette clé dérivée.
5. Envoie le mot de passe et les paramètres d'Argon2id au serveur en les chiffrant avec la clé publique du serveur.

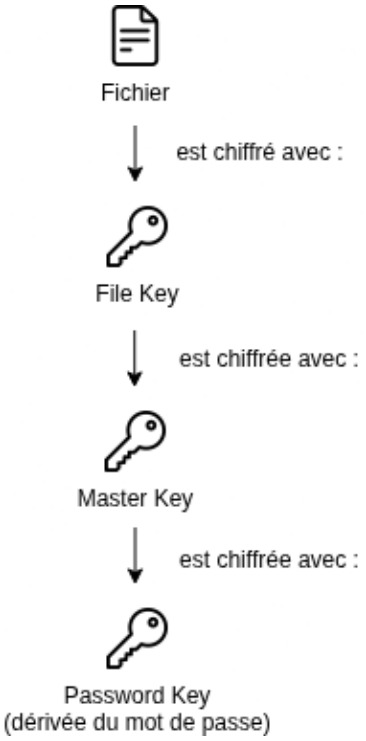


Fig. 2. – Étapes de chiffrement des fichiers.

Une fois le chiffrement effectué, la `Master Key` chiffrée est stockée dans un fichier à la racine du dossier et les fichiers chiffrés de l'utilisateur ont la forme :

```
File Key chiffrée || Nonce || Tag || Données chiffrées
```

La Fig. 3 ci-dessous nous montre quelle entité possède quelles informations après le chiffrement des fichiers.

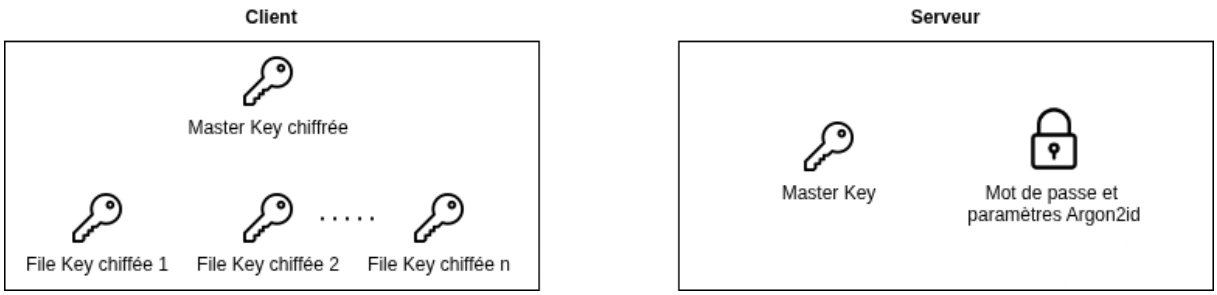


Fig. 3. – Informations stockées après le chiffrement des fichiers.

## 1.4. Paiement de la rançon

Lors du choix de l'option `Pay` :

1. Le serveur envoie le mot de passe et les paramètres d'Argon2id au client.
2. Le client dérive la clé à partir du mot de passe et des paramètres d'Argon2id reçus, puis déchiffre la `Master Key` avec AES-KW en utilisant cette clé dérivée.
3. Le client déchiffre chaque `File Key` avec AES-KW en utilisant la `Master Key`.
4. Le client déchiffre chaque fichier avec AES-GCM en utilisant la `File Key` correspondante.

## 1.5. Déchiffrement d'un fichier spécifique

Lors du choix de l'option `Unlock one file` :

1. Le client envoie la `File Key` chiffrée au serveur en la chiffrant avec la clé publique du serveur.
2. Le serveur déchiffre le message reçu avec sa clé privée, déchiffre la `File Key` avec AES-KW en utilisant la `Master Key` et l'envoie au client.
3. Le client déchiffre le fichier avec AES-GCM en utilisant la `File Key` reçue.

## 1.6. Changement de mot de passe

Lors du choix de l'option `Change password` :

1. Le client obtient un mot de passe aléatoire d'un dictionnaire, le chiffre avec la clé publique du serveur et lui envoie.
2. Le serveur déchiffre le message reçu avec sa clé privée, dérive une clé avec Argon2id à partir du mot de passe reçu, chiffre la `Master Key` avec AES-KW en utilisant cette clé dérivée et l'envoie au client.
3. Le client remplace l'ancienne `Master Key` chiffrée par la nouvelle dans le fichier à la racine du dossier.

## 1.7. Niveau de sécurité choisi

Le ransomware utilise le niveau de sécurité **V**, qui offre une sécurité au moins aussi forte que AES256.

## 1.8. Algorithmes utilisés

### 1.8.1. Chiffrement symétrique

Pour le chiffrement des fichiers, l'algorithme **AES256-GCM** est utilisé avec les paramètres suivants :

- Taille de la clé : 256 bits.
- Taille du nonce : 96 bits.
- Taille du tag : 128 bits.

Ces paramètres permettent de chiffrer des fichiers d'une taille maximale d'environ 68 GB.

Pour le chiffrement des clés (`Master Key` et `File Key`), l'algorithme **AES-KW** est utilisé avec une taille de clé de 256 bits. Cet algorithme permet de ne pas avoir à stocker de nonce et de tag tout en protégeant les clés.

#### 1.8.2. Chiffrement asymétrique

Pour le chiffrement asymétrique, l'algorithme post-quantique **Kyber1024** est utilisé avec les paramètres suivants :

- Taille de la clé publique : 1568 bytes.
- Taille de la clé privée : 3168 bytes.

Cet algorithme permet de garantir une sécurité au moins aussi forte que AES256.

#### 1.8.3. Dérivation de clé

Pour dériver la `Master Key` à partir du mot de passe, l'algorithme **Argon2id** est utilisé avec les paramètres suivants :

- Taille du sel : 16 bytes.
- Taille de la clé dérivée : 32 bytes.
- Nombre d'itérations : 1.
- Degré de parallélisme : 4.
- Coût en mémoire : 65536 KB.

Une taille de clé dérivée de 32 bytes permet d'obtenir une `Master Key` compatible avec AES256.

### 1.9. Spécificités

#### 1.9.1. Pourquoi l'architecture est résistante aux attaques post-quantiques ?

#### 1.9.2. Pourquoi le niveau de sécurité V est le même partout ?

#### 1.9.3. Qu'est-ce qui permet au ransomware d'être sûr que le mot de passe est légitime ?